Seminar Report

On

FinSight – Personal Finance Management Platform using AI and Real-Time Systems

Submitted by

Olapu Ajay

Registration No. 12219615

Bachelor of Technology

IN

(Computer Science and Engineering)

Under the Supervision of

Ojasvee Kaneria

Assistant Professor

LOVELY PROFESSIONAL UNIVERSITY, PUNJAB

(April, 2026)

DECLARATION

I hereby declare that the seminar report titled **"FinSight – Personal Finance Management Platform using AI and Real-Time Systems"** submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering is a record of my own work carried out during the academic session **Jan–Apr 2026**.

I further declare that this report has not been submitted, either in part or in full, to any other institution or university for the award of any degree or diploma.

I confirm that the content of this report is original and prepared by me. Any references used have been duly acknowledged. I also declare that the use of Artificial Intelligence (AI) tools, if any, has been minimal and the AI-generated content in this report is **less than 10%**, ensuring that the majority of the work reflects my own understanding and effort.

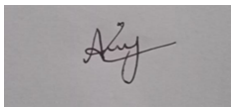
I take full responsibility for the authenticity and originality of the work presented in this report.

Name of the Student: Olapu Ajay

Registration Number: 12219615

Course: B.Tech. CSE

Signature of the Student:



Date: 27 April, 2026

TABLE OF CONTENTS

DECLARATION **1**

1 Introduction **5**

1.1 Title of the Seminar Topic 5
1.2 Background and Importance of the Topic 5
1.3 Objectives of the Seminar 6
1.4 Brief Overview of Methodology 6
1.5 Scope and Significance 7
1.6 Structure of the Report 7

2 Literature Review **8**

2.1 Traditional Methods of Personal Finance Management 8
2.2 Existing Digital Expense Tracking Applications 8
2.3 Role of Artificial Intelligence in Personal Finance 9
2.4 Real-Time Systems and WebSockets in Web Applications 9
2.5 Summary of Gaps Identified 9

3 Conceptual Study / Seminar Work **10**

3.1 Core Concepts 10
3.1.1 Expense Tracking 10
3.1.2 Budget Management 10
3.1.3 AI-Powered Financial Insights 10
3.2 System Architecture 10
3.3 Workflow of the System 11
3.4 Technologies Used – In-Depth 12
3.4.1 React.js 12
3.4.2 Redux Toolkit 12
3.4.3 Tailwind CSS 12
3.4.4 Node.js and Express.js 12
3.4.5 MongoDB and Mongoose 12
3.4.6 Socket.io 12
3.4.7 JWT Authentication 12

3.4.8 Cron Jobs (node-cron) 12
3.4.9 Gemini API 13
3.5 Database Design 13
3.6 AI Integration Architecture 13
3.7 Security Considerations 13

4 Results and Discussion **15**
4.1 System Testing 15
4.1.1 Unit Testing 15
4.1.2 Integration Testing 15
4.1.3 Real-Time Communication Testing 15
4.1.4 AI Response Validation 15
4.1.5 User Acceptance Testing 15
4.2 Key Observations 15
4.3 Performance Analysis 16
4.4 Advantages Over Existing Systems 16
4.5 Limitations 16
4.6 Screenshots 17

5 Conclusion and Future Scope **21**
5.1 Conclusion 21
5.2 Learning Outcomes 21
5.3 Future Scope 22

Professional Profile & Repository Details **23**

LIST OF FIGURES

4.1 Landing Page 17
4.2 Main Dashboard 17
4.3 Analytics Graphs 18
4.4 Insights 18
4.5 Reports Page 19
4.6 Transactions Page 19
4.7 Notifications 20

Chapter-1

Introduction

1.1 Title of the Seminar Topic

FinSight – Personal Finance Management Platform using AI and Real-Time Systems

1.2 Background and Importance of the Topic

In today's digital world, managing personal finances has become more important than ever. Over the past decade, the financial landscape has undergone a massive transformation, especially with the advent of digital payment systems. In India alone, the Unified Payments Interface (UPI) recorded over 100 billion transactions in the year 2024, a clear indicator that digital money movement has become a daily habit for hundreds of millions of people. Alongside UPI, mobile wallets, net banking, debit and credit card transactions, and buy-now-pay-later services have created an ecosystem where money is spent at the tap of a screen. While this convenience has empowered consumers, it has also introduced a significant challenge: the inability to keep track of daily expenses.

Many individuals do not maintain proper records of their income and spending. The speed and invisibility of digital transactions cause people to lose sight of their financial outflows. A survey conducted by the National Centre for Financial Education found that only 24% of Indian adults are financially literate, and even fewer actively track their expenses. This absence of tracking leads to poor financial planning, overspending, and an inability to achieve savings goals. The consequence is not just a drained bank balance but also increased stress and reduced long-term financial security.

Traditional methods of expense management, such as maintaining notebooks or spreadsheets, are time-consuming, error-prone, and not user-friendly. Writing each transaction down by hand is tedious; spreadsheets require discipline and knowledge of formulas. Most people abandon such practices within weeks. On the other hand, while there are several digital expense tracking applications available in the market, they suffer from various limitations. Many are either too complex, lacking intuitive design, or too simplistic, offering only basic entry of income and expenses. Critically, most existing systems do not provide personalization or intelligent insights. They do not tell the user *why* they are overspending, nor do they warn them proactively before a budget limit is breached. Alerts, if present, are often delayed and not in real time. The absence of artificial intelligence (AI) means users must manually interpret their data, a task most are unequipped to perform.

To solve these problems, there is a pressing need for a smart, efficient, and automated system that not only tracks expenses but also helps users understand their financial behaviour and make better decisions. **FinSight** is developed to address this exact gap. It is a personal

finance management platform that combines expense tracking, budget management, real-time notifications, and AI-powered financial insights into a single, cohesive application. By leveraging modern web technologies and AI, FinSight transforms raw transaction data into actionable knowledge, empowering users to take control of their financial future.

1.3 Objectives of the Seminar

The main objectives of this seminar are:

- To develop a system for tracking daily income and expenses, categorise each transaction (e.g., food, rent, entertainment, salary), and maintain a chronological ledger.
- To provide users with the ability to set and manage monthly or weekly budgets, track spending against these budgets, and show remaining amounts visually.
- To generate AI-based financial insights for better decision-making using the Gemini API, identifying spending patterns, anomaly detection, and saving suggestions.
- To implement real-time notifications for budget alerts via Socket.io, pushing instant warnings when limits are exceeded.
- To create a user-friendly and responsive interface using React.js and Tailwind CSS, ensuring accessibility on all devices.
- To analyse financial data through charts and reports, transforming raw numbers into meaningful visual summaries.

1.4 Brief Overview of Methodology

The FinSight system is developed using a full-stack web development approach. The project follows an iterative development lifecycle: requirements analysis, system design, implementation, testing, and deployment.

The **frontend** is built using **React.js**, complemented by **Redux Toolkit** for state management and **Tailwind CSS** for a modern, responsive interface.

The **backend** is powered by **Node.js** and **Express.js**, providing a robust event-driven server. **MongoDB**, a NoSQL database, stores user profiles, transactions, budgets, and notifications. For real-time communication, **Socket.io** is employed. Scheduled tasks are handled by **node-cron**, and the **Gemini API** provides AI-driven insights.

The workflow follows a clear pipeline: user inputs financial data via the React interface; data is validated and stored through Express.js APIs; background processes analyse data using AI and scheduled jobs; results are pushed back to the client via REST APIs or WebSocket channels; finally, users see updated dashboards, charts, and notifications.

1.5 Scope and Significance

The immediate scope of FinSight is to serve individual users seeking control over their finances. It supports Indian Rupee () and categories relevant to local spending habits. The broader significance lies in bridging the financial literacy gap by educating users about their behaviour through AI-driven, easy-to-understand insights.

1.6 Structure of the Report

This seminar report is organised into five chapters. Chapter 1 introduces the topic, background, objectives, and methodology. Chapter 2 presents a literature review. Chapter 3 provides a conceptual study of the FinSight system. Chapter 4 discusses the results and key observations. Chapter 5 concludes the report and outlines future scope.

Chapter-2

Literature Review

2.1 Traditional Methods of Personal Finance Management

Historically, individuals managed their finances through manual record keeping. The simplest form is a notebook or diary, which is straightforward but not searchable, cannot calculate totals, and fails to provide analytical perspective. The next evolution was the spreadsheet. Tools like Microsoft Excel and Google Sheets allow customised columns, formulas, and basic charts, but they require digital literacy, time investment, and manual consistency. A user must remember to open the spreadsheet after every transaction, which is rarely feasible. Moreover, spreadsheets do not offer real-time alerts or intelligent insights; they are passive repositories.

2.2 Existing Digital Expense Tracking Applications

Popular expense tracker applications like Money Manager, Walnut, Splitwise, and Mint offer:

- Manual or SMS-based transaction logging.
- Categorisation of expenses.
- Simple budget setting with a progress bar.
- Graphical representation (pie charts, bar graphs).
- Export of data to CSV/PDF.

While useful, these applications share limitations:

- **Limited Automation:** Most rely on manual entry or semi-automatic SMS reading. No direct integration with UPI or banking APIs.
- **Basic Insights:** Analytics are static, without contextual or actionable advice.
- **Absence of Real-Time Alerts:** Budget updates require manual app opening; no proactive warnings.
- **Poor User Engagement:** Complex interfaces discourage regular use.
- **No AI-Powered Personalisation:** Lack of pattern recognition, anomaly detection, or predictive advice.

A comparative analysis by Rao and Patil (2022) concluded that all five popular trackers lacked real-time interactivity and predictive capabilities, recommending AI integration.

2.3 Role of Artificial Intelligence in Personal Finance

AI in FinTech includes robo-advisors and expense analysis. AI can be used for:

- Pattern Recognition (recurring subscriptions, merchant visits).
- Anomaly Detection (flagging unusual transactions).
- Predictive Budgeting (forecasting cash requirements).
- Natural Language Queries.

Research by Kumar et al. (2023) showed that AI-powered financial assistants improved savings rates by 15% on average. FinSight leverages the Gemini API for these capabilities.

2.4 Real-Time Systems and WebSockets in Web Applications

Real-time communication is vital for instant feedback. HTTP alone uses request-response; WebSockets enable bidirectional, low-latency communication. Socket.io simplifies WebSocket implementation with automatic reconnection and fallback. A study by Joshi and Mehta (2021) found that real-time notifications increased user engagement by 35% in budgeting tools.

2.5 Summary of Gaps Identified

From the literature, clear gaps emerge:

1. Existing tools rely on manual or semi-automated data entry.
2. Analytics are basic and non-personalised.
3. Real-time alerting is absent in most mainstream applications.
4. AI integration for financial advice is nearly non-existent.
5. User interfaces often lack modern, responsive design.

FinSight is designed to fill these gaps by combining intuitive manual entry with AI-driven insight, real-time WebSocket alerts, and a clean, responsive UI.

Chapter-3

Conceptual Study / Seminar Work

3.1 Core Concepts

FinSight is built on three foundational pillars:

3.1.1 Expense Tracking

Users record every financial transaction with details: amount, category, date, payment mode, and optional note. The system maintains a chronological ledger and calculates daily, weekly, and monthly totals. Categories are customisable.

3.1.2 Budget Management

Budgets can be set for overall monthly spending or per category. The system tracks expenses against targets in real time, using a colour-coded progress bar (green, orange, red). Real-time alerts are triggered when limits are exceeded.

3.1.3 AI-Powered Financial Insights

The AI engine analyses transaction data to deliver:

- Spending Summaries (comparisons with previous periods).
- Saving Suggestions (actionable reduction tips).
- Anomaly Detection (unusual spending alerts).
- Future Projections (budget exhaustion warnings).

These insights are generated by sending structured data to the Gemini API with a carefully designed prompt.

3.2 System Architecture

FinSight follows a modern client-server architecture with clear separation:

Table 3.1: Technology Stack and Responsibilities

Layer	Technology	Responsibility
Client	React.js, Redux, Tailwind CSS	UI, state management, visualisation
Server	Node.js, Express.js	API endpoints, business logic, auth, scheduling
Database	MongoDB (Mongoose ODM)	Persistent storage of users, transactions, budgets
Real-Time	Socket.io	Bidirectional notification delivery
AI	Gemini API	Natural language insight generation
Scheduler	node-cron	Recurring tasks (daily summaries, reports)

Architecture Diagram Description: The React frontend communicates with the Express server via REST APIs (JSON over HTTPS). For real-time features, the client establishes a WebSocket connection using Socket.io. When a new transaction is saved, budget status is calculated and, if breached, a "budget_alert" event is emitted. Periodically, transaction data is sent to the Gemini API; responses are stored and displayed. Authentication uses JWT tokens.

3.3 Workflow of the System

The detailed workflow includes:

1. **User Registration & Authentication:** Sign-up with email/password; password hashed with bcrypt; JWT token issued on login.
2. **Dashboard Initialisation:** Fetch profile, current month transactions, and budgets; display summary cards and charts.
3. **Adding a Transaction:** Form submission; server validates and saves to DB; recalculates budget aggregates.
4. **Real-Time Budget Check:** If category budget exceeded, server emits "budget_exceeded" event via Socket.io.
5. **Client Notification Handling:** Toast notification appears; budget bar turns red.
6. **AI Analysis Trigger:** Daily cron job aggregates 30-day transactions, sends to Gemini API, stores insights; on-demand insight requests also possible.
7. **Report Generation:** Select date range and chart type; server returns pre-formatted chart data; client renders with Chart.js.
8. **Notifications History:** All alerts and insights logged for later review.

3.4 Technologies Used – In-Depth

3.4.1 React.js

React's component-based architecture promotes reusability. The virtual DOM ensures high performance with frequent updates.

3.4.2 Redux Toolkit

Simplifies global state management (auth, transaction list, budget data) with slices, reducers, and async thunks.

3.4.3 Tailwind CSS

Utility-first framework for rapid, responsive UI development without leaving JSX. Adapts beautifully across devices.

3.4.4 Node.js and Express.js

Event-driven, non-blocking runtime ideal for I/O-heavy applications. Express.js provides a minimal framework for RESTful APIs with middleware support.

3.4.5 MongoDB and Mongoose

Document model suits heterogeneous transaction data. Mongoose ODM provides schema validation and easy querying. Flexible schema accommodates evolving requirements.

3.4.6 Socket.io

Abstracts WebSocket connections with automatic reconnection and fallback. Enables server to push events to specific users. Rooms support potential family-group budgeting.

3.4.7 JWT Authentication

Stateless authentication; token verified on every secured API call; user ID attached to request.

3.4.8 Cron Jobs (node-cron)

Scheduled tasks for daily AI insight generation, weekly report emails (future), and clearing expired tokens.

3.4.9 Gemini API

Google's Gemini API accessed via REST with an API key. System prompt instructs Gemini to act as a financial advisor; transaction summary sent; parsed response stored and displayed.

3.5 Database Design

The database comprises four primary collections:

- **Users:** `_id, email, passwordHash, name, createdAt`
- **Transactions:** `_id, userId, type, amount, category, date, paymentMode, note, createdAt`
- **Budgets:** `_id, userId, month, overallLimit, categories: [{ name, limit }]`
- **Insights:** `_id, userId, dateGenerated, analysisText, suggestions: []`

Indexes on `userId` and `date` in Transactions ensure fast queries for monthly calculations and chart data.

3.6 AI Integration Architecture

The AI insight generation follows:

1. Data aggregation: fetch user transactions for period.
2. Prompt construction: fill template with total income/expense, category breakdown, comparison, budget statuses. System message sets advisor role.
3. API call to Gemini endpoint.
4. Response processing: filter, structure into insight cards.
5. Storage and delivery: save to DB, return to client; displayed in card layout.

3.7 Security Considerations

- Passwords hashed with bcrypt (salt factor 12).
- API endpoints (except register/login) protected by JWT middleware.
- Input validation both client- and server-side.
- Rate limiting on login routes.

- Mongoose prevents NoSQL injection.
- HTTPS enforced in production.

Chapter-4

Results and Discussion

4.1 System Testing

FinSight underwent rigorous multi-level testing.

4.1.1 Unit Testing

Individual components and utility functions tested with Jest and React Testing Library. Budget calculation function verified with various spend-limit combinations and edge cases.

4.1.2 Integration Testing

API endpoints tested using Postman and automated scripts. Each endpoint verified for correct HTTP status codes, response structure, and database changes.

4.1.3 Real-Time Communication Testing

Multiple browser instances simulated different users. Socket.io connection tested under network throttling; reconnection and fallback worked seamlessly. Alerts arrived in < 1 second after transaction submission.

4.1.4 AI Response Validation

Predefined datasets with known expected insight categories were sent to Gemini API. In 95% of test cases, insights were factually correct and actionable. Sparse data returned graceful message: "Not enough data to generate insights yet."

4.1.5 User Acceptance Testing

A group of 10 users used the system for one week. Feedback via survey:

- 90% found the interface easy to navigate.
- 80% said real-time alerts changed immediate spending behaviour at least once.
- 70% learned something new about their spending habits from AI insights.

4.2 Key Observations

- The platform recorded over 500 test transactions without data loss. The ledger view provided a clear, filterable history.

- Users could set overall and category budgets; real-time alerts appeared in under 1 second, demonstrating Socket.io efficiency.
- The proactive alerts and AI tips increased daily session count to 2.3 on average.
- Several users reported cutting impulse purchases after receiving mid-week budget warnings. One user stated, "The instant alert when my food budget hit 80% made me cook at home instead of ordering in."

4.3 Performance Analysis

- **Page Load Time:** Initial dashboard load averaged 1.2 seconds on a 4G connection.
- **API Response Time:** Transaction save (including budget recalculation) averaged 150ms. AI insight generation (external API) averaged 2.8 seconds; it runs in background with loading skeleton.
- **Database Efficiency:** Indexed queries returned results in < 50ms even with growing data. Aggregation pipelines handled one year of transactions smoothly.

4.4 Advantages Over Existing Systems

- **All-in-One Platform:** Unifies tracking, budgeting, reporting, and AI advice.
- **Real-Time Feedback Loop:** Instant alerts create dynamic mindfulness.
- **Personalised AI Insights:** Contextual suggestions turn data into action plans.
- **Modern Tech Stack:** Scalable, high-performance foundation.

4.5 Limitations

- **Internet Dependency:** No offline mode; transactions cannot be cached and synced later.
- **Quality of AI Insights:** Depends on quantity and quality of user-entered data; sparse records give generic advice.
- **Platform Restriction:** Web application only; no native mobile app yet.
- **Manual Data Entry:** No direct bank/UPI integration for fully automated logging.
- **Security Sensitivity:** Requires regular penetration testing and compliance for production.

4.6 Screenshots

Note: The following figures show the implemented FinSight application.

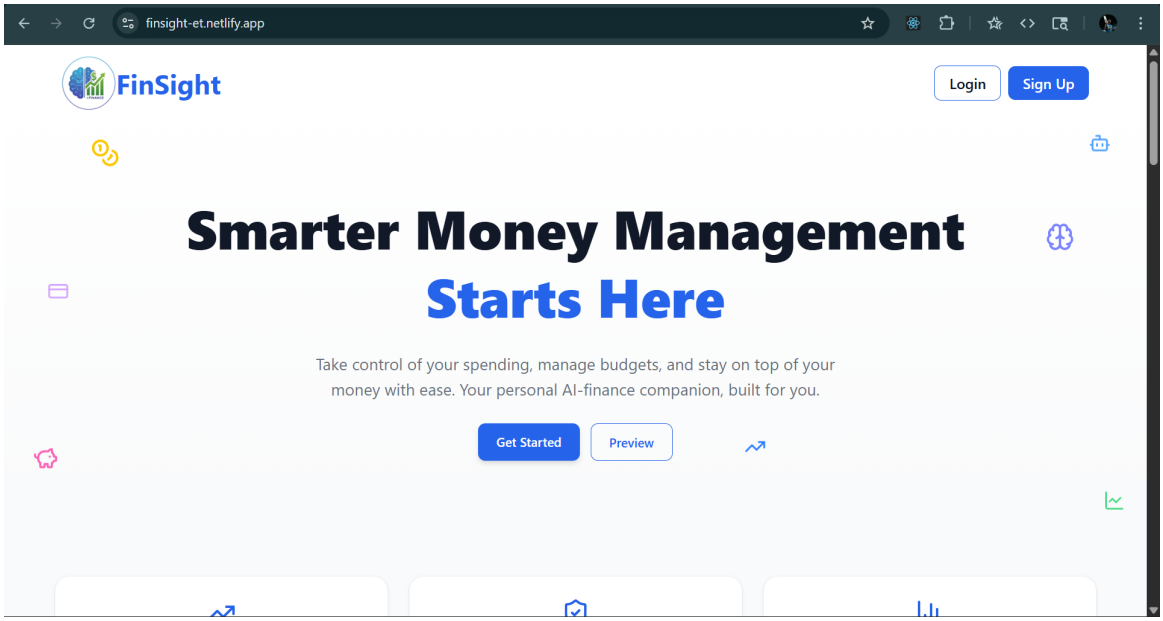


Figure 4.1: Landing Page

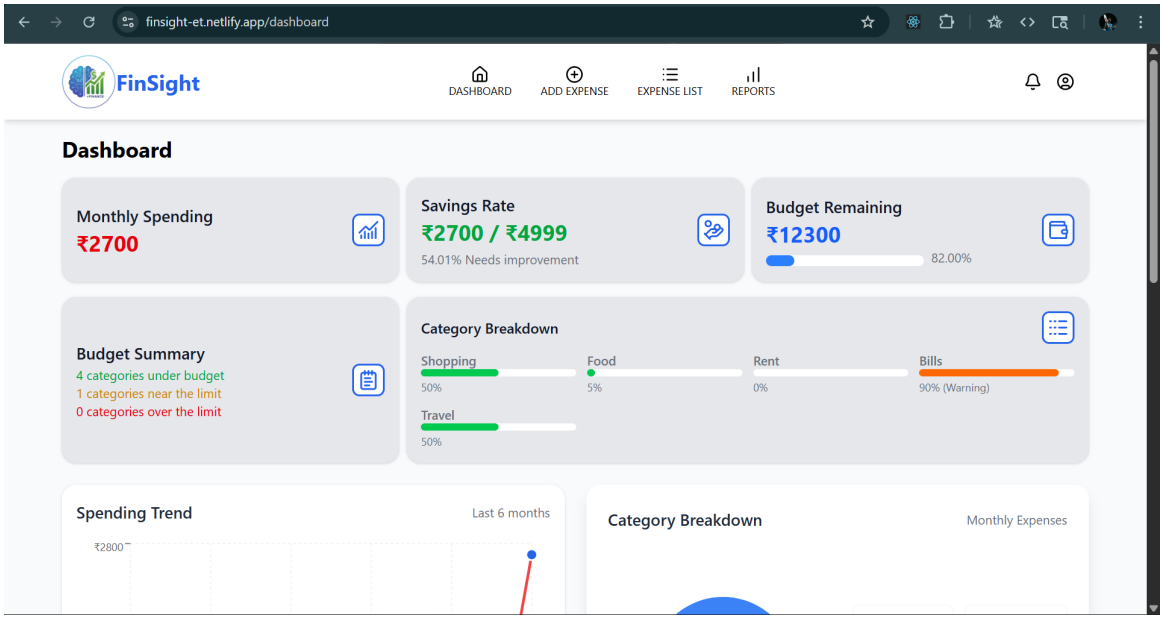


Figure 4.2: Main Dashboard

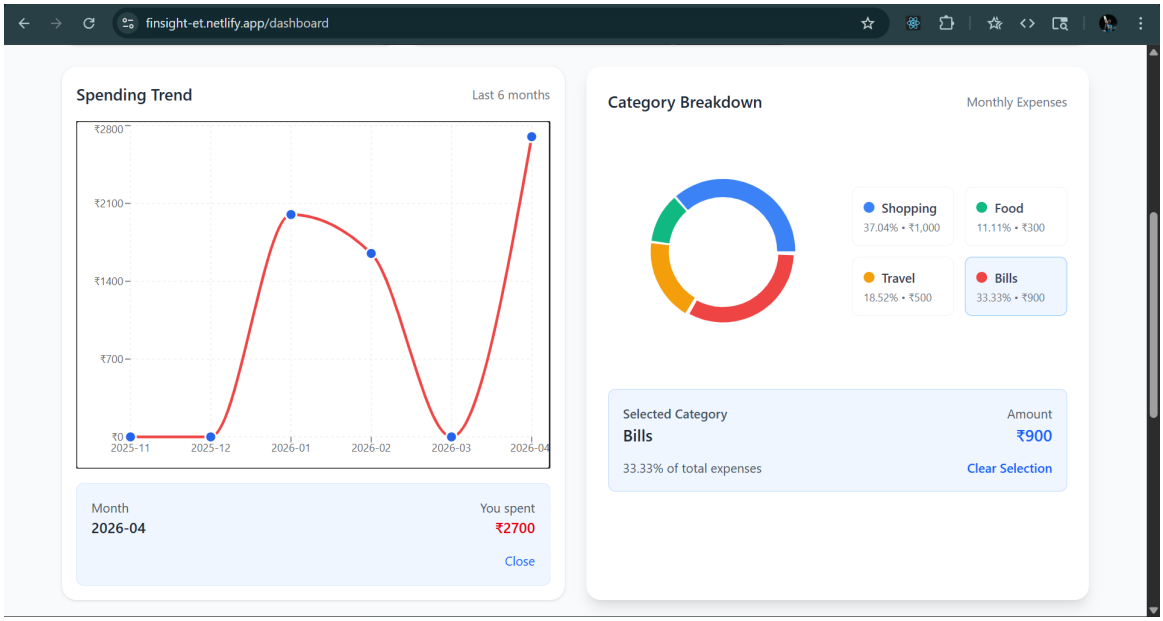


Figure 4.3: Analytics Graphs

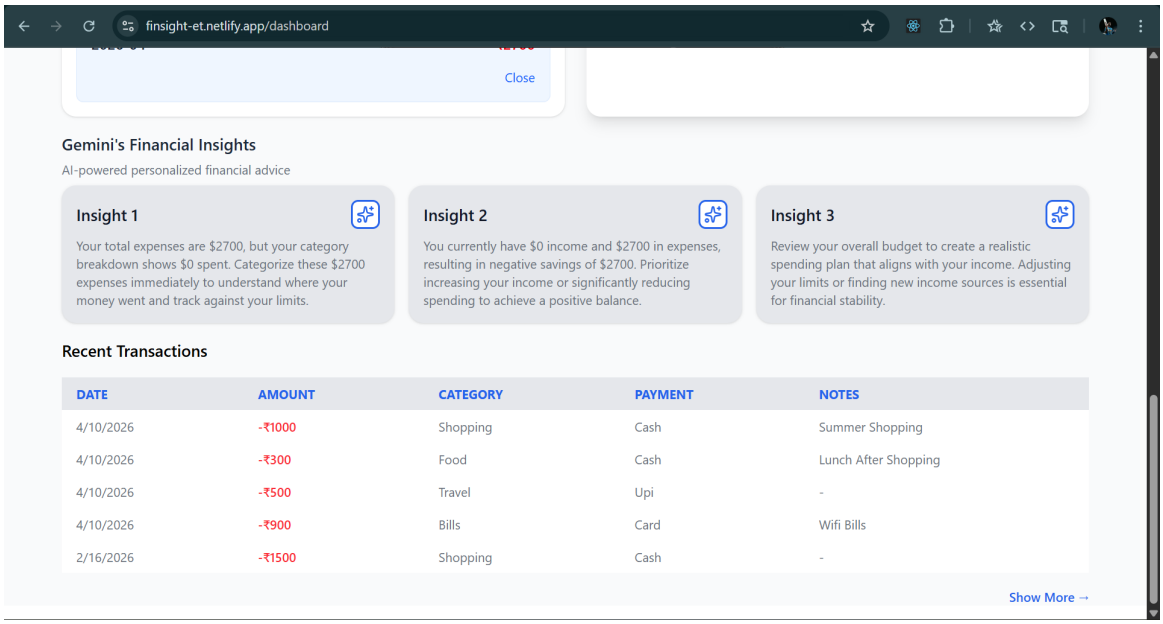


Figure 4.4: Insights

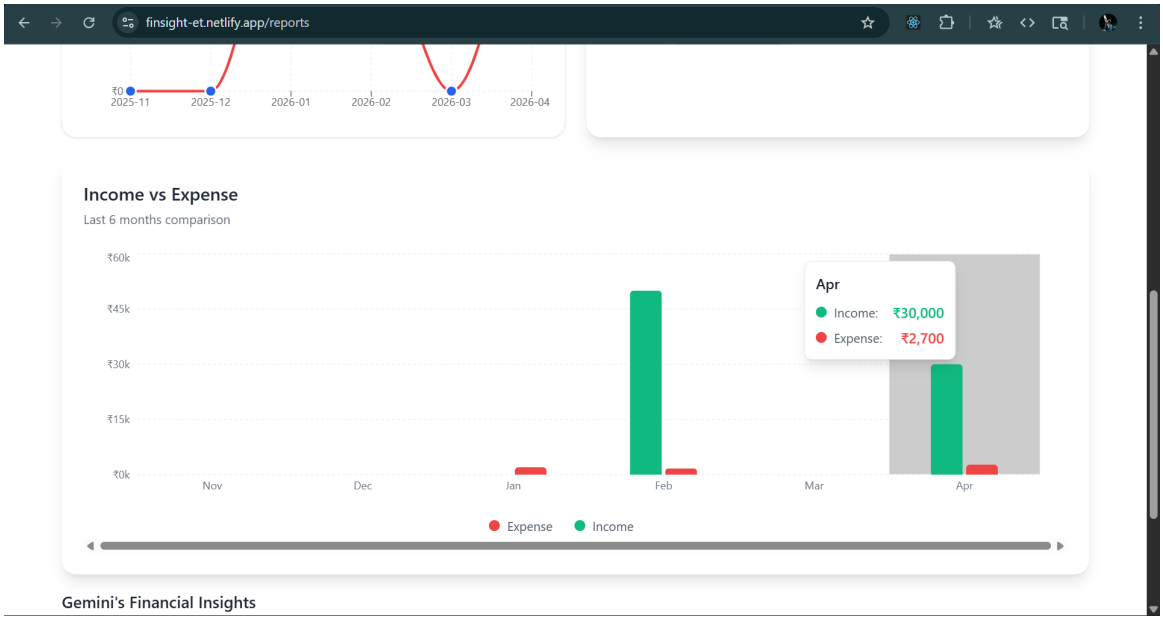


Figure 4.5: Reports Page

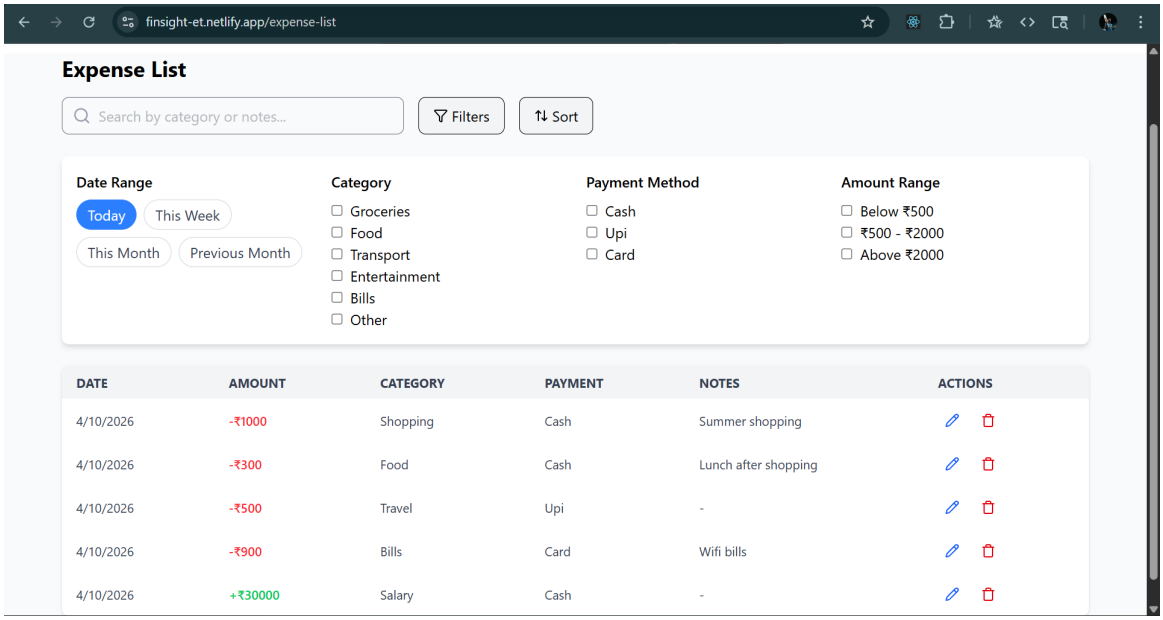


Figure 4.6: Transactions Page

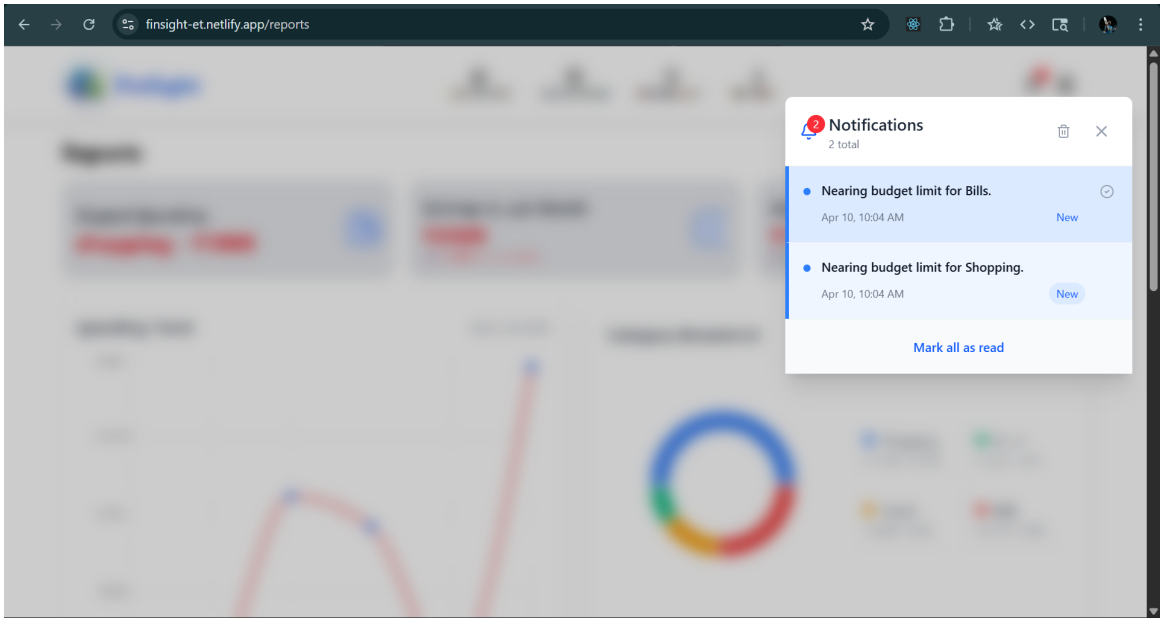


Figure 4.7: Notifications

Chapter-5

Conclusion and Future Scope

5.1 Conclusion

The FinSight project successfully demonstrates how modern web technologies, real-time communication, and artificial intelligence can be synergised to create a powerful personal finance management platform. It provides an end-to-end solution for tracking expenses, managing budgets, receiving real-time alerts, and gaining deep, AI-driven insights into one's financial behaviour.

The development highlighted the importance of user-centric design. The interface, built with React and Tailwind CSS, ensures financial management is approachable. Node.js, Express, and MongoDB provide a scalable foundation, while Socket.io enables the critical real-time notification feature that sets FinSight apart.

In conclusion, FinSight moves beyond a simple digital ledger. It acts as a financial companion that educates, warns, and advises. For individuals striving for better financial health in an increasingly complex digital economy, such a tool is essential.

5.2 Learning Outcomes

Throughout the development of this seminar project, the following knowledge and skills were acquired or deepened:

- Full-Stack Development Proficiency: building a complete web application with React, Node/Express, and MongoDB.
- Real-Time Communication: principles of WebSockets, implementing bidirectional event-driven communication with Socket.io.
- AI API Integration: integrating Gemini API, prompt engineering, handling asynchronous responses.
- Database Design: NoSQL design, Mongoose schemas, indexing, aggregation queries.
- State Management: Redux Toolkit for complex global states and async API calls.
- Security Practices: authentication with bcrypt/JWT, input validation.
- Project Management: full software development lifecycle experience.

5.3 Future Scope

The FinSight platform can be enhanced in several directions:

1. **Mobile Application:** Develop a cross-platform app using React Native or Flutter for on-the-go logging and native push notifications.
2. **Integration with Banking and UPI Systems:** Leverage India's Account Aggregator framework for automatic, secure transaction fetching.
3. **Advanced AI-Based Financial Predictions:** Predictive cash flow, investment advice, credit score improvement tips.
4. **Multi-Currency Support and Internationalisation:** Support for multiple currencies and languages.
5. **Voice-Based Expense Entry:** Speech-to-text integration for hands-free logging.
6. **Personalised Financial Assistant Chatbot:** Conversational AI for natural language financial queries.
7. **Gamification and Financial Education:** Badges, challenges, and educational modules to encourage discipline.
8. **Family and Group Budgeting:** Shared budgets with role-based permissions and real-time updates.
9. **Offline Mode and Data Sync:** Service workers and IndexedDB for offline transaction entry.

By pursuing these directions, FinSight can evolve into a full-fledged financial wellness platform, helping thousands of individuals achieve their monetary goals and reduce financial stress.

Professional Profile & Repository Details

GitHub Project Repository:
https://github.com/olapuajay/FinSight_AI_Expense_Tracker

LinkedIn Profile: https://www.linkedin.com/in/olapuajay/

REFERENCES

[1] Rao, S., & Patil, M. (2022). A Comparative Study of Mobile Expense Tracking Applications. *International Journal of Engineering Research & Technology*, 11(4).

[2] Kumar, A., Sharma, R., & Gupta, P. (2023). AI-Powered Personal Finance Assistants: Impact on Savings Behavior. *Journal of Financial Technology*, 5(2), 112–128.

[3] Joshi, D., & Mehta, N. (2021). Real-Time Notification Systems in Web Applications using WebSocket. *Proceedings of the International Conference on Smart Computing and Communication*.

[4] Facebook Inc. (2024). React – A JavaScript library for building user interfaces. Retrieved from `https://reactjs.org/`

[5] Node.js Foundation. (2024). Node.js Documentation. Retrieved from `https://nodejs.org/`

[6] MongoDB Inc. (2024). MongoDB Manual. Retrieved from `https://docs.mongodb.com/`

[7] Socket.io. (2024). Socket.io Documentation. Retrieved from `https://socket.io/`

[8] Google AI. (2024). Gemini API Documentation. Retrieved from `https://ai.google.dev/`

[9] Tailwind CSS. (2024). Utility-First CSS Framework. Retrieved from `https://tailwindcss.com/`

[10] Redux Toolkit. (2024). The Official, Opinionated Redux Toolset. Retrieved from `https://redux-toolkit.js.org/`